

One-Month ML Learning & Project Plan (Stock Price Prediction with PyTorch)

Main idea is here: <https://medium.com/swlh/stock-price-prediction-with-pytorch-37f52ae84632>

Executive summary

Phase 1: Basics

Learn core ML concepts, Python & Pandas fundamentals (1–2 weeks).

Phase 2: Build

Implement a PyTorch LSTM/GRU model for stock price prediction (1–2 weeks).

Phase 3: Finish

Evaluate model performance, compare LSTM vs GRU, document project (1–2 weeks).

Program Objective: *Over ~4–6 weeks of full-time study, a beginner with no prior ML experience will learn fundamental machine learning concepts and Python data tools, then develop a regression-based time series project predicting stock prices using PyTorch (implementing LSTM & GRU models). By the end, the participant will have a working Jupyter Notebook, a well-structured GitHub repository with code and README, basic model evaluation metrics (e.g. MSE/RMSE), and a comparison of LSTM vs. GRU results.*

Below is a weekly breakdown with main focus areas, activities, and expected outcomes:

Week	Focus & Activities	Key Outcome by End of Week
1	ML basics (supervised vs unsupervised, regression vs classification), Python basics, introduction to Jupyter & Pandas (part 1)	Understanding of ML fundamentals; Python & Jupyter environment set up; basic data manipulation skills.
2	Continue ML basics (regression, model evaluation), Pandas & data analysis (part 2), simple regression mini-exercise	Knowledge of regression & evaluation metrics; proficiency in Pandas for data wrangling; completed a simple linear regression exercise.

3	PyTorch fundamentals (quickstart tutorial), intro to time series forecasting & RNN concepts (LSTM/GRU)	Working knowledge of PyTorch (tensors, training loop); conceptual understanding of RNNs and sequence modeling.
4	Project implementation: data preparation (stock data, scaling, sliding window), building LSTM & GRU models in PyTorch	Own code for data preprocessing and initial LSTM/GRU model definitions; ready to train models on stock data.
5	Project training & testing: fit LSTM & GRU models, tune hyperparameters, track metrics	Trained LSTM and GRU models; initial evaluation metrics (MSE/RMSE); identified any model overfitting or issues.
6	Model evaluation & comparison, final improvements, documentation & wrap-up	Final model evaluation (compare LSTM vs GRU performance); cleaned code in Jupyter Notebook; prepared README and documentation for GitHub.

Phase 1: ML & Data Science Fundamentals (Weeks 1–2)

Goal: Build foundational knowledge in machine learning concepts and the basic tools (Python, Jupyter, Pandas) needed for data analysis and ML. This phase corresponds to **Weeks 1–2** of the program.



Advanced Participants

Note: If a participant already knows ML basics, they may skip Phase 1 and directly start with the PyTorch tutorial (*Temel ML bilenler için bir PyTorch örneği*).

Week 1: Introduction to ML Concepts & Python. Set up the learning environment and cover core ML ideas.

- **Day 1: Setup and Orientation:** Install necessary tools (e.g. Python, Jupyter Notebook via Anaconda or similar) and ensure the **PyTorch library** is installed. If the participant lacks coding background, spend time on a **Python fundamentals primer** (data types, control flow, writing basic scripts). The IBM course “*Python Basics for Data Science*” is recommended as a prerequisite. Begin exploring Jupyter Notebook basics (creating notebooks, running simple Python commands). [\[edx.org\]](https://www.edx.org)
- **Day 2: What is Machine Learning?** Read Google’s “**What is ML?**” introduction to grasp the high-level idea of letting models learn patterns from data. Learn key

terminology (dataset, model, training, prediction) and the distinct types of ML systems (supervised vs. unsupervised). [\[developers...google.com\]](#), [\[developers...google.com\]](#) [\[developers...google.com\]](#)

- **Days 3–5: IBM Machine Learning Course – Modules 1 & 2:** Start the **IBM Machine Learning with Python** online course (IBM ML0101EN) for a structured intro to ML. Cover *Module 1: Introduction to ML* (applications, **supervised vs. unsupervised learning**, common ML libraries in Python) and *Module 2: Regression*. By end of Week 1, the participant should be able to **explain supervised vs unsupervised learning, and understand what regression means** as a prediction of numeric values. They should also know that ML models require evaluation (e.g. knowing what a train/test split and metrics like RMSE are). [\[edx.org\]](#) [\[developers...google.com\]](#)

Week 2: Regression, Data Handling, and Pandas. Continue learning ML fundamentals with a focus on **regression** (since our project is a time-series regression problem) and build up **data analysis skills** with Pandas.

- **Day 6: IBM Course – Modules 2 & 3:** Finish *Module 2: Regression* content (e.g. linear vs non-linear regression, basic model evaluation methods) and move into *Module 3: Classification* for broad knowledge (KNN, decision trees, logistic regression, etc.). Pay attention to how model evaluation is done (train/test split, concept of **mean squared error** (MSE) and **root mean squared error** (RMSE) as a performance measure), since **our final project will use MSE/RMSE** to gauge accuracy. [\[edx.org\]](#)
- **Day 7–8: Python Data Analysis (Pandas):** Work through a **Pandas tutorial** to become proficient in data manipulation. Use the W3Schools Pandas guide (or an equivalent, like a GeeksforGeeks introduction) to learn how to load datasets (e.g. `pd.read_csv`), inspect DataFrames, handle missing values, filter/select data, and compute basic statistics. Practice by loading a small sample dataset (CSV file) in a Jupyter Notebook and performing simple operations (view top rows, get summary info, basic plots if possible). By doing this, the participant **reinforces Python skills and gets comfortable with Pandas**, which will be vital for prepping stock data in the project. [\[geeksforgeeks.org\]](#), [\[geeksforgeeks.org\]](#)
- **Day 9–10: Regression Mini-Exercise:** Solidify understanding of regression with a small hands-on exercise. For example, use **Scikit-learn** to fit a simple linear regression on a toy dataset (e.g. predicting house prices or car mileage from an online dataset). This can be done in a Jupyter Notebook and should include splitting data into train/test sets, training a linear model, predicting on the test set, and computing **MSE or RMSE** for evaluation. This mini-project ensures the participant can connect theory to practice: preparing data, training a model, and evaluating results in code. By the end of Week 2, **the participant should have a firm grasp of**

basic ML terms and a working knowledge of Python and Pandas to handle datasets.
[\[edx.org\]](https://edx.org)

Milestone (end of Phase 1): The participant understands fundamental ML concepts (supervised vs unsupervised, regression vs classification) and can perform simple data analysis in Python. **Output:** Completed introductory ML coursework (IBM course, Google ML intro), a few basic Jupyter notebooks with Pandas practice, and a simple regression exercise with results.

Phase 2: PyTorch & Time Series Project Implementation (Weeks 3–4)

Goal: Transition from theory to practice by learning PyTorch and beginning the stock price prediction project. The participant will acquire *deep learning* basics and start building LSTM/GRU models for time series forecasting.

Week 3: PyTorch Fundamentals & RNN Concepts. Gain familiarity with the PyTorch deep learning framework and understand recurrent neural networks (RNNs), the foundation for LSTM/GRU models.

- **Day 11–12: PyTorch Quickstart Tutorial:** Follow the official PyTorch “**Learn the Basics**” Quickstart tutorial. This hands-on tutorial introduces the essential **PyTorch workflow** – from **tensors and datasets to building a simple model, training loop, and evaluating performance**. The participant should practice writing PyTorch code (e.g., using `torch.Tensor`, `Dataset` and `DataLoader` for data, defining a simple neural network with `nn.Module`, running a training loop, and calculating loss). The PyTorch tutorial uses a simple dataset (like Fashion-MNIST) for demonstration, which is fine – the goal is to become comfortable with **PyTorch syntax and training process** (data loading, forward pass, loss computation, backpropagation, optimizer step).
[\[docs.pytorch.org\]](https://docs.pytorch.org)
- **Day 13: Intro to Time Series & RNNs:** Read about **time series forecasting** and **Recurrent Neural Networks** conceptually. For example, the participant can revisit the *RNN explanation* in the Medium article by Rodolfo Saldanha, which defines time series problems and introduces RNN architecture and variants LSTM and GRU. Key takeaways should include: a time series is sequence data indexed by time, and RNNs (like LSTM/GRU) are neural networks designed to capture sequential dependencies by maintaining hidden state (memory) over time steps. Understand *why* we use LSTMs/GRUs for stock prediction: **stock prices over time form a sequence where past values may influence future values**, and RNN-based models can learn these temporal patterns. [\[medium.com\]](https://medium.com), [\[medium.com\]](https://medium.com) [\[medium.com\]](https://medium.com)
- **Day 14–15: Plan the Project Approach:** Before coding, outline the steps needed for the stock prediction project:

- o Choose a **dataset**: The example references use 12+ years of Amazon stock prices from a public dataset. The participant can use the **DJIA 30 Stock Time Series** data from Kaggle (as in Rodolfo's example), focusing on one stock (e.g. AMZN or any one company) for simplicity. [\[medium.com\]](#)
- o **Data Preparation Strategy**: Identify how to prepare data for an RNN. From the example, note that we need to extract the relevant column (e.g., "Close" price) and normalize it (the Medium example uses MinMax scaling to [-1,1] range). Plan to implement a **sliding window method**: using a rolling window of past N days to predict the next day, as illustrated by the reference code which constructs sequences (lookback=20) for training. Sketch the model structure: we will implement two RNN models (an **LSTM** and a **GRU**), each taking sequences of length N (e.g. 20 days of prices) as input and predicting the next day's price. [\[medium.com\]](#)
- o **Environment Check**: Ensure PyTorch is correctly installed and verify you can load the chosen stock dataset using Pandas (small test: read a CSV of stock prices and plot or inspect it).

By the end of Week 3, the participant should have **completed a PyTorch beginner tutorial** (so they are confident writing and running PyTorch code) and have a clear plan for how to implement the stock price prediction model (data prep, model, training approach). They should conceptually understand RNNs (LSTM/GRU) and why they're suitable for the task. [\[medium.com\]](#)

Week 4: Project Implementation – Data Prep & Modeling. Begin coding the actual project in a Jupyter Notebook, focusing on preparing the data and defining the LSTM/GRU models in PyTorch.

- **Day 16–17: Data Collection & Preparation**: Obtain the stock price data. For example, the **Kaggle notebook by Rodolfo Saldanha** provides a pointer to a dataset of historical stock prices – the participant should download or otherwise access similar data (e.g., CSV file of daily prices). Load the data using Pandas (`pd.read_csv`), focusing on one stock's data for the project. Perform **exploratory data analysis**: visualize the closing price over time (matplotlib line plot) to see trends. Then apply **data preprocessing** as planned: [\[medium.com\]](#)
 - o Extract the relevant feature, e.g., the "Close" price column. [\[medium.com\]](#)
 - o Normalize the series, e.g. using `sklearn.preprocessing.MinMaxScaler` to scale prices between -1 and 1 as in the example code. [\[medium.com\]](#)
 - o Implement the **sliding window** function to create input sequences (e.g. 20 days of prices) and corresponding targets (the 21st day price) from the time series. At this point, split the sequences into a **training set and test set** (for

instance, 80% train, 20% test as in the example code). Ensure to shuffle or randomize if appropriate, or if preserving sequence order, just take the first 80% of the sequences as training and the rest as test. [\[medium.com\]](#)

- o Convert the prepared numpy arrays into **PyTorch tensors** (e.g., using `torch.from_numpy`) because PyTorch models require tensor inputs. [\[medium.com\]](#)
- **Day 18–19: Define LSTM & GRU Models in PyTorch:** Using PyTorch's `nn.Module`, define two model classes: **LSTMModel** and **GRUModel** (or similar names). Each should be a small RNN with a few layers. Refer to PyTorch's official RNN docs if needed, but the Medium article actually provides a blueprint of an LSTM class in PyTorch and a similar GRU class, both using 2 layers with a certain hidden size (e.g., 32 units) and one output neuron. While writing this code in the notebook, **explain each component** with comments for learning purposes (e.g., “the LSTM holds an internal state, we initialize hidden state h_0 and cell state c_0 to zeros for each sequence in the batch”). [\[medium.com\]](#), [\[medium.com\]](#) [\[medium.com\]](#)
 - o Set common hyperparameters: input dimension = 1 (since we use one feature: price), output dimension = 1 (predicting a single value), hidden layer size (e.g., 32 or 50 units), number of layers (e.g., 2), and number of epochs for training (e.g., start with 50–100 epochs as in the reference). [\[medium.com\]](#), [\[medium.com\]](#)
 - o Define the **loss function** (use Mean Squared Error, `nn.MSELoss`, since this is a regression problem) and an **optimizer** (e.g. Adam or SGD). [\[medium.com\]](#), [\[medium.com\]](#)
- **Day 20: Begin Training Loop:** Write the training loop for one of the models (say LSTM first) to ensure it works. At each epoch, forward-pass the training data, compute loss, backpropagate (`loss.backward()`), and update weights (`optimizer.step()`). Print the epoch number and training loss for monitoring convergence. Verify that the loss decreases and the code runs without errors. This day might be iterative: adjusting shapes, debugging PyTorch errors (common for beginners), and ensuring the model is learning (loss going down). [\[medium.com\]](#), [\[medium.com\]](#) [\[medium.com\]](#)

By the end of Week 4, the participant should have a **functioning codebase**: the data is preprocessed and loaded into tensors, and the LSTM/GRU model definitions and training loop are in place. They might have initial training results for the LSTM model (and ready to implement for GRU in Week 5). The Jupyter Notebook (or scripts) will be the main working area for this.

Phase 3: Model Training, Evaluation & Documentation (Weeks 5–6)

Goal: Complete model training, evaluate and compare the performance of LSTM vs GRU, and finalize all project deliverables (code, results, documentation).

Week 5: Model Training and Evaluation. Run full training for both models and analyze their performance.

- **Day 21–22: Train LSTM & GRU Models:** Finish any remaining implementation and train both models on the prepared dataset. Likely, the LSTM was partially trained in week 4; ensure it's fully trained for the planned number of epochs. Then instantiate the **GRU model** with the same hyperparameters and run training on the same data (the training loop code will be almost identical for GRU). Track the training time for each model (e.g., using Python's `time` module) and ensure the models aren't overfitting (monitor the loss for signs of divergence or if it plateaus early).
[\[medium.com\]](#) [\[medium.com\]](#), [\[medium.com\]](#)
- **Day 23: Evaluate Model Performance:** After training, use both models to **generate predictions** on the test set (unseen data). Compute evaluation metrics:
 - **MSE / RMSE:** Calculate the Mean Squared Error of each model's predictions vs actual prices on the test set (since both LSTM and GRU outputs are continuous values). Optionally, take the square root of MSE to get RMSE (more interpretable in original units). [\[medium.com\]](#)
 - **Comparison:** Compare the LSTM and GRU results. For example, **Rodolfo Saldanha's project found that GRU slightly outperformed LSTM, achieving a lower MSE on the test set and finishing training ~5 seconds faster**. Note if your results align with expectations (GRUs often have fewer parameters and can train faster). Also, plot the predicted vs actual stock prices for both models to visualize accuracy (line plot of actual vs predicted prices over time for the test period) – this can reveal if the models captured the trend or overfitted to training data (e.g., look for divergence in forecast lines).
[\[medium.com\]](#), [\[medium.com\]](#)
- **Day 24: Hyperparameter Tuning (if needed):** If initial results are poor (e.g., very high error or obvious overfitting), allocate time to experiment with improvements. Possibilities: adjust the lookback window length (e.g., 10 days vs 20 days), tweak number of epochs or hidden units, or add regularization (like Dropout layers, as some Keras example did with 20% dropout after each RNN layer). Only moderate tuning is expected given the timeframe; the goal isn't to get the perfect model but to practice the iterative process of improvement.

Week 6: Project Conclusion – Documentation & Presentation. Finalize the project by cleaning up code, documenting, and reflecting on lessons learned.

- **Day 25–26: Code Refactoring & Comments:** Review the Jupyter Notebook code and **refactor** if necessary. Ensure the notebook is organized logically (e.g., sections for **Data Loading, Data Preprocessing, Model Definition, Training, Evaluation**, etc., with Markdown explanations in between code). Remove any test code or irrelevant outputs to produce a clean, **runnable notebook** for the final repository. Write **clear comments** in the code explaining key steps (especially for the model and training loop) so that others (and your future self) can understand it.
- **Day 27: Documentation (README & Report):** Create a **README.md** for the GitHub repository. This should include an overview of the project (objective and dataset description), instructions on how to set up and run the code, and summarize results. Also, prepare any additional documentation needed – for instance, a short **report or presentation** highlighting the approach and findings (if required, this can be in the form of the Jupyter Notebook’s markdown or a separate document). Summarize key results (e.g., *“GRU model achieved RMSE of X vs LSTM’s Y on test data”*), and include any charts (like the actual vs predicted price plot) with brief analysis.
- **Day 28–30: Final Review & Critical Thinking:** Double-check that all deliverables are complete:
 - o Ensure the **Jupyter Notebook runs from top to bottom without errors**, producing the intended results and plots.
 - o The **GitHub repository** is structured (perhaps folders for data, notebooks, and scripts, and the README at root) and contains the necessary files (notebook, data or data reference, etc.).
 - o The **README** explains how to use the project, and possibly insight into the results and how LSTM vs GRU performed.
 - o The **evaluation metrics** and comparison of models are clearly presented (e.g., a small table or section in the notebook comparing LSTM and GRU MSE/RMSE and training time). Finally, take a step back to reflect.
 Encourage the participant to read a critical perspective on AI, such as *“AI Snake Oil”* by Narayanan & Kapoor. This reading (or the authors’ blog posts) will help them understand the *limitations and ethical considerations of ML*, emphasizing critical thinking. For instance, stock price prediction is notoriously hard and often yields limited real-world accuracy – being aware of hype versus reality is valuable for a balanced learning experience.
 [press.princeton.edu](https://press.princeton.edu/books/hardcover/9780691249131/ai-snake-oil)

Milestone (end of Phase 3): The project is fully implemented and documented. Both LSTM and GRU models have been trained on historical stock data and evaluated (with metrics like

MSE/RMSE). The participant can articulate which model performed better and why (e.g., “GRU achieved slightly lower test error and faster training time than LSTM in our experiments”). All deliverables are ready to share or present. [\[medium.com\]](https://medium.com)

Final Deliverables & Checklist

By the end of this one-month program, the new ML practitioner will have produced:

- **Jupyter Notebook:** A clean, **fully runnable notebook** containing the code for data loading, preprocessing (Pandas), model definitions (PyTorch LSTM & GRU), training loops, and evaluation. The notebook should be interwoven with Markdown explaining the steps, findings, and any issues encountered (for self-documentation and easier understanding for others).
- **GitHub Repository:** A well-organized repository hosting the project code and assets. This typically includes the Jupyter Notebook (and possibly a script version of the model code), sample data or a link to the data source (since stock data file might be large), and the documentation files. Ensure the repository has a clear structure (e.g., directories for data, notebooks, results, and any modules).
- **README Documentation:** A comprehensive **README.md** at the root of the repository describing the project. It should outline the project’s **purpose** (stock price time-series regression with LSTM/GRU), the **dataset** (source and description of the stock data), how to **run the code** (environment setup and execution steps), and a summary of **results** (including metrics like training and testing MSE/RMSE for each model, and possibly a brief comment on why one model performed better). This serves as both a user guide and an executive summary of the work.
- **Model Evaluation & Comparison:** Documented evaluation of both models. Include a **comparison of LSTM vs GRU performance** on the test data (e.g., tabulate their MSE or RMSE, and note training time differences). Provide one or two plots (e.g., actual vs predicted price for each model) to visualize model predictions. Discuss potential reasons for the performance difference, referring to known characteristics (e.g., “GRU often trains faster due to fewer parameters, which we also observed”). [\[medium.com\]](https://medium.com)
- **Reflection & Next Steps:** (Optional) A short section in the notebook or README where the participant notes what they learned and potential next steps (e.g., “try more features or more advanced models like Transformers for time series” or “explore why stock prediction is challenging”). Including a nod to critical thinking (one idea: mention insights from *AI Snake Oil* about not overestimating predictive power of ML in complex domains). [\[press.princeton.edu\]](https://press.princeton.edu)

1 Notebook

Clean, runnable, well-commented Python code with ML model training

2 GitHub Repo

Organized project repository (code, data link, results, docs)

3 README

Overview, setup instructions, project findings (LSTM vs GRU results)

4 Metrics

MSE/RMSE metrics for model performance, with a brief comparison

5 Plots

Visualization of actual vs predicted stock prices for both models

Following this comprehensive plan, a complete beginner will gradually build up from the very basics of machine learning to developing a working stock price prediction model with PyTorch. By structuring the learning into **clear phases and weekly milestones**, the participant can monitor progress and ensure they achieve both theoretical understanding and practical implementation skills. The guided use of high-quality resources (IBM and Google ML introductions, Pandas tutorials, PyTorch official guides, and project examples from Medium/Kaggle) provides a solid learning path. In the end, the participant will not only have a finished project to showcase but also a stronger grasp of machine learning workflows, time series analysis, and the importance of evaluating models critically with real data. [\[medium.com\]](https://medium.com), [\[press.princeton.edu\]](https://press.princeton.edu)